OC

QA Test Strategy

TLDR - QA Strategy

The main goal is to make sure we don't lose the quality we achieved during GLS as the product continues to change. We'll use the golden companies as our regression baseline, validate calculations exactly, and test AI outputs separately for quality and consistency. We also want to automate the most important checks so we can confidently support the wider partner rollout.

1. Purpose and Sprint 5 context

Sprint 5 is a housekeeping and knowledge-transfer sprint rather than a normal feature-delivery sprint. The team is using the workshops to understand architecture, calculations, UI/Chat, agents, BFF/case services, CI/CD, database and migrations before the larger post-GLS backlog is prioritised and sized.

Sprint 5 QA direction QA should use Sprint 5 to understand how Opportunity Catalyst actually works, convert the workshop knowledge into reusable regression coverage, and prepare the test model for the next delivery phase. QA should not continue investing in the current GLS Stage environment unless the team specifically asks for it.

What this strategy is for

- Define how QA will test the existing Opportunity Catalyst journey: Target -> Peers -> Case for Change -> Size of Prize -> Output, with Chat available throughout the journey.
- Define how QA will test the post-GLS backlog items already identified for Target, Peers, Analysis/Size of Prize and Chat.
- Use the architecture workshop to test the correct technical layer underneath each user flow: Experience BFF, services, data, real-time events and AI agents.
- Reuse the eight seeded golden companies and known expected values as a stable regression baseline while the future production data model is being prepared.
- Keep the known defects found during the previous QA cycle as permanent regression scenarios.

Out of scope for this document

- Detailed acceptance criteria for backlog items that have not yet been written or agreed.
- Final production performance/SLA thresholds where the team has not yet defined a target.
- Detailed Business Unit expected behaviour until the new Business Unit stories define the ownership and rules.

2. Product under test - Opportunity Catalyst

Opportunity Catalyst is a guided analysis journey. QA should always think about the whole flow because a change in an earlier step can change later calculations, narratives and exported output.

Step	What the user does	Main system behaviour	Main QA concern
Target	Select a company and set up the analysis.	Company Search resolves the company and the case stores the selected target/context.	Search correctness, search limits, target data, refresh persistence and safe no-result/error handling.
Peers	Review proposed peers, add/remove peers, open peer rationale and confirm the set.	Peer Curation and related agents build the peer set; real-time updates show progress; peer data drives later benchmarks.	Self-peer exclusion, duplicates, loading state, pagination, comparability, stale state, confirmation timing and persistence.
Case for Change	Review/rank/select the narrative framings that explain why action is needed.	The application combines analysis and generated framing to build the business story.	Correct evidence, selection state, generated wording, single-click navigation and downstream consistency.
Size of Prize	Review profit/cash opportunities, drill into buckets/levers and apply overrides/selections.	Calculation rules combine peer gaps, BAIN experience, selected levers and working-capital logic.	Exact calculations, range/open-ended rules, overrides, rounding, recalculation, BU metrics and state persistence.
Output	Generate the final deck/export and executive narrative.	Latest confirmed values are converted into slides, charts, tables and narrative.	Reconcile numbers to Size of Prize, verify deck layout, labels, footer, overflow and PPT/HTML export.
Chat	Ask questions about the current analysis and, post-GLS, attach sources/change screen state.	Chat uses data/services/agents to answer questions and will increasingly interact with the active workflow.	Grounding, source correctness, calculation explanation, hallucination, chat-to-screen state changes and audit/evidence.

3. Architecture context - only what QA needs

The workshop clarified the technical path underneath the product. QA does not need to turn every ticket into an infrastructure test, but it does need to know which layer owns the behaviour being tested.

Simple request flow Browser/UI -> Microsoft Entra -> Azure Front Door -> Private Link -> Experience BFF -> specialist services -> PostgreSQL / Blob / Redis / events / AI agents -> response back to UI

Component	What it means in Opportunity Catalyst	How QA uses this information
Experience BFF	The frontend mainly calls the BFF. The BFF coordinates the specialist services.	When a UI value is wrong, verify whether the BFF response is wrong, incomplete or stale before treating it as only a frontend defect.
Case Service	Owns cases/opportunities and case state.	Validate create/update/persist/refresh, concurrent changes and correct case lifecycle.
Company Search	Resolves company names/tickers using normalised/fuzzy search.	Test exact/partial/ticker/alias/no-result searches, result limits and backend pagination.
Peer Curation	Owns the propose-peer journey and triggers peer-related agents.	Test peer creation, target exclusion, duplicate prevention, pending analysis, final peer set and downstream recalculation.
Real-time service	Streams progress to the UI using server-sent events.	Test Agent Working -> final score, reconnect/retry, duplicate/out-of-order events and whether Confirm is gated correctly.
Audit + Evidence	Audit records what happened; Evidence records why/source/override reasoning.	For overrides, chat actions and important decisions, verify both the action and the reason/source can be traced.
AI agents / Foundry	Use-case-specific agents are called by services; model calls are expected through Portkey.	Test business answer plus tool/source path, timeouts, bad tool data, model/gateway failure and grounding.
PostgreSQL / Blob / Redis / Service Bus	Different stores support transactional data, documents/search artefacts, current GLS golden/demo data and async events.	Validate the correct persistence/data path for the feature instead of treating the backend as one database.

Detailed Azure/network architecture is useful reference material, but it is not the main body of this test strategy. QA will use it only where a story touches authentication, routing, service connectivity, events, agents or environment migration.

4. QA objectives for Opportunity Catalyst

Objective	What success looks like in this product
Protect the end-to-end journey	A target/peer change must not silently create a wrong Case for Change, Size of Prize or Output later.
Protect calculation correctness	Peer gaps, BAIN experience, lever selection, working-capital calculations, rounding and open-ended values must follow the agreed rule engine exactly.
Protect state integrity	Refresh, autosave, multiple users, multiple tabs, real-time updates and delayed requests must not overwrite newer valid user changes.
Protect source/decision traceability	Values and AI explanations should be traceable to the correct source, evidence, override or decision where the product exposes that information.
Protect AI quality	Peer rationale, chat answers, recommendations and generated narrative should be accurate, grounded and consistent with the deterministic numbers.
Protect generated deliverables	The final deck/PPT/HTML should use the latest values and remain visually usable with no overflow, missing labels or footer collisions.
Scale beyond manual checking	Use golden cases and automated data checks so QA does not manually validate every company when the dataset expands.

5. Sprint 5 / Post-GLS backlog coverage

The following checks are tied directly to the current OC backlog. They are proposed QA coverage and should be refined when the actual story and acceptance criteria are written.

Target screen

Planned change	What it means in the app	What QA will check
Search limits	Very broad company searches need a controlled result size/fallback instead of returning an unusable list.	Exact, partial, ticker, broad and no-result searches; confirm the configured limit, fallback message and no raw error.
Refresh persistence	Target/options should come back from backend state instead of depending on temporary localStorage.	Change values, refresh/reopen case, compare API vs UI, verify no stale browser state overwrites server state.
Header Share button	The missing Share action will be added to the creation workflow header.	Visibility, basic action, layout and any defined sharing behaviour. Do not invent permission/link rules unless the story defines them.

Peers module

Planned change	What it means in the app	What QA will check
Real-time agent loading	The UI needs a dedicated loading state while peer agents are still analysing.	Watch from initial loading through Agent Working/Not Scored to final score; confirm failure/retry behaviour and correct Confirm gating.
Backend search pagination	Add Peer search moves from frontend filtering to backend paging.	First/next page, query changes, stable order, no duplicates across pages, target exclusion and already-selected peer exclusion.
Business Unit logic	Business Unit relationships/workflows are currently missing and will be introduced.	Once ACs exist: BU selection/mapping, allowed peers, downstream metric impact and missing/unknown BU. Keep expectations open until ownership/rules are agreed.
Data source drilldown	Peer data sources and adjustments will be visible in drilldown.	Correct source, adjustment, lineage, missing-source handling and correct source-specific rendering.
Peer sorting	Sorting changes from alphabetical only to alphabetical + recently added.	Initial order, newly added position, refresh persistence and no duplicate/disappearing peers.
Revenue by Region / Analyst View / CAPIQ graph / Confidence %	New peer-detail content is being added.	Visibility conditions, values vs source data, CAPIQ-only rendering, empty states and confidence percentage consistency.

Analysis / Size of Prize

Planned change	What it means in the app	What QA will check
Business Unit metrics	BU-level metrics will be shown inside Size of Prize.	Reconcile BU values to source data and verify BU changes only affect the intended calculations/lever outputs.
Add Lever	Users will be able to add a lever from the bucket drilldown.	Add/remove, duplicate prevention, recalculation, persistence, audit/evidence and Output reconciliation.
Recommended Actions + Sized Opportunity	Sub-lever detail will expose actions and opportunity values more directly.	Exact numeric values, source lineage, generated recommendation consistency and no contradiction between recommendation and calculation.

Chat

Planned change	What it means in the app	What QA will check
+ Source attachment	User can attach a source from the Chat input.	Attach/remove, supported/unsupported source, grounding to the selected source, missing-source failure and source visibility in the answer.

Chat-to-screen workflow	Chat can edit/convert an instruction into active screen state.	Exact intended change, confirmation/guardrail, persistence, audit/evidence, recalculation and rollback/undo only if defined.
UI polish	Visual/layout issues in Chat will be cleaned up.	Overflow, responsive layout, keyboard behaviour, loading/empty/error states and separation of cosmetic vs functional findings.

AI safety and agent-action checks

- Test attached documents that contain hidden or malicious instructions and confirm the agent does not follow them.
- Verify Chat actions only use declared and reversible tools, run in the requesting user's context, and are audited as actions performed by the agent on the user's behalf.
- Add prompt-injection and unsafe tool-use scenarios to the AI regression set.

6. Deterministic and AI testing - specific to OC

Area	Examples from OC	How QA evaluates it	Best automation layer
Deterministic	Peer count; selected peers; target exclusion; BAIN/peer ranges; opportunity totals; working capital; rounding; API schema; state persistence; exported values.	Exact expected result. Any mismatch is a functional defect unless an approved rule change explains it.	API/service/data first; critical UI flow second.
AI / non-deterministic	Peer rationale; Case for Change framing; Chat answer; recommended actions; executive/deck narrative.	Score accuracy, grounding, completeness, relevance, consistency and hallucination. Do not require exact wording.	Evaluation harness + golden questions; targeted manual review.
Mixed	Peer score + rationale; Size of Prize number + explanation; Chat answer explaining a calculation; Output narrative using deterministic values.	Assert the numbers/state exactly, then score the generated explanation separately.	Service/API assertions + AI eval.

AI evaluation set to keep

- Questions that ask for exact opportunity values and bucket breakdowns for the seeded companies.
- Questions asking how a value was calculated, including open-ended ranges and peer-vs-BAIN precedence.
- Questions asking for source/lineage: Capital IQ, BAIN experience, selected peer/benchmark and override reason.
- Questions where the relevant lever is switched off, to ensure Chat does not present it as an active opportunity.
- Paraphrased questions to check whether the answer remains consistent without expecting identical wording.
- Run each key golden question multiple times to check consistency, not only with paraphrased wording.
- Re-run the AI evaluation set whenever a prompt, model or agent tool changes, and before a release.
- Keep a small BCN-reviewed golden subset as the reference set for release decisions.
- Where existing eval.contract.json files are suitable, reuse that format so the evaluation criteria stay close to the agent implementation.

7. Golden data and large-dataset strategy

The current seeded GLS dataset is already valuable because QA has manually reconciled many values and behaviours. It should stay as the baseline even after the current Stage environment is retired.

Golden company	Why keep it	Representative QA use
Adobe	Best-known reference case with detailed Size of Prize, working-capital and Chat expectations.	Calculation rule regression, open-ended values, source lineage, Chat explanation, Output reconciliation.
Carrier	Different peer/chart/output profile.	Deck rendering, chart labels, Data Coverage layout regression, deterministic output checks.
General Mills	Known USD case with multiple benchmark/productivity/cash examples.	Range calculations, cost/productivity charts, preview-vs-export rendering.
Beiersdorf	EUR case and clean Data Coverage example.	Currency regression and deck-layout control case.
Graphic Packaging	Known case with deck differences and Data Coverage overlap.	Rendering regression and comparison against reference deck behaviour.
Dow	Different industrial/chemical peer set.	Peer-selection diversity, calculations, chart/deck regression.
Adidas	EUR consumer case.	Currency/sector diversity and deck-layout regression.
Honda	JPY case with very large values.	JPY formatting, large-number calculations, working capital and output regression.

Golden-set expansion The future production scope discussed in post-GLS planning includes US-based companies in Healthcare (MedTech) and Advanced Manufacturing & Services / auto-parts suppliers. When live/reference data is available, add representative companies from those sectors rather than replacing the current eight golden cases.

Golden regression as the stage-to-main safety net

As GLS functionality moves from the BFF into specialist services, run the golden regression before and after the change. The expected calculations, key user-flow behaviour and generated outputs should remain consistent unless there is an approved requirement change.

How to handle larger datasets

1. Run automated validations across the full dataset: null/missing fields, duplicates, invalid ranges, currency/period mismatch, impossible values, missing sources and calculation-rule violations.
2. Run the full golden regression on the eight seeded companies.
3. Flag statistical/business anomalies and failed contract checks.
4. Manually review the golden set plus the anomalies, not every company.
5. For AI outputs, run the standard evaluation question set and manually investigate low-scoring answers.

8. Contract, integration and real-time testing

Contracts

The architecture workshop highlighted centrally managed JSON schemas. For Opportunity Catalyst, malformed service data can directly break later screens, so contract checks should be a normal regression gate.

- Validate required fields and types for BFF responses and service events used by Target, Peers, Case for Change, Size of Prize and Chat.
- Keep a regression for a 200 response that is structurally invalid, because this previously exposed a raw React Router/framework error.
- Verify correlation/causation IDs where the contract includes them so a request can be traced across services/events.
- Verify invalid events fail safely and follow the agreed dead-letter/error path instead of corrupting the case state.

Real-time / asynchronous behaviour

Scenario	Expected Opportunity Catalyst behaviour
Add peer while analysis is still running	Existing selected peers stay intact; new peer shows pending state; Confirm remains blocked until required analysis is ready.
Older peer-set read returns after a new Add Peer	Older response must not remove the newly added peer or replace the latest revision.
Duplicate Service Bus/SSE event	UI/service remains idempotent; counts and state do not duplicate.
Refresh while operation is processing	Case returns to a consistent server-backed state with clear loading/retry behaviour.
Same user edits the same case in two tabs	Updates from one tab must not be silently overwritten by stale state from the other tab; the latest valid server-backed revision must win or a clear conflict state must be shown.
Temporary network loss	User sees a controlled failure/retry state; no half-confirmed or duplicate action is created.
Two users work on the same case	Real-time updates should propagate to both users without losing valid changes. Conflicting edits must follow the agreed server-side conflict/versioning rule and must not silently overwrite newer state.

9. Permanent regression risks from the previous QA cycle

These are not generic examples. They are behaviours already found in Opportunity Catalyst and should stay in regression until the design changes make them irrelevant.

Risk / previous finding	Area	Regression check	Why it matters
Malformed 200 response exposed raw framework error	Frontend contract / Peers	Return missing/invalid required peer data and verify a safe product error state.	Prevents user-facing stack traces/crashes when backend data is malformed.
Target company could appear as its own peer	Peer Service / Peers	Search/add target company and verify it cannot be stored or confirmed as a peer.	Self-peer corrupts benchmark calculations.
Confirm while added-peer analysis pending	Peers / Peer Changes	Add peer, keep analysis pending and verify Confirm is blocked.	Prevents locking incomplete peer state.
Comparability score and band mismatch	Peers	Compare numeric/derived score with user-facing band after recalculation.	Avoids contradictory peer-quality messages.
Stale read overwrote newer Add Peer	Peers / async state	Force older read to return after newer change and verify latest state wins.	Protects user changes from disappearing.
Duplicate peer storage/count mismatch	Peer Service/events	Propose duplicate peers and compare stored members with completion count/event.	Keeps UI/service/event counts consistent.
Open-ended + dropped in preview	Size of Prize	Keep an open-ended lever active and verify bucket/total preview/output retain +.	Prevents understating an open-ended opportunity.
Data Coverage footer overlap	Output/deck	Generate representative decks and check footer collision/overflow in browser and exported PPT.	Protects Bain template conformance.
Skip this step required repeated clicks	Case for Change navigation	Single click should immediately trigger processing/disabled state and navigate once.	Prevents duplicate clicks and unclear navigation feedback.

10. Automation strategy for this application

Automation should be placed where the behaviour is easiest to prove reliably. The goal is not to automate every screen; it is to protect the calculations, state and service boundaries that drive the product.

Automation layer	What to automate in Opportunity Catalyst	Examples
Contract / schema	BFF/service request-response and event schemas used by the main journey.	Missing required peer field, invalid max_peers type, case event schema, search artefact contract.
Service / API	Business rules, state transitions, duplicate handling, concurrency, persistence and failures.	Peer target exclusion, dedup count, confirm gating, Size of Prize calculations, stale revision protection.
Data validation	Golden expected values and source-to-calculation reconciliation.	Adobe opportunity values, Honda JPY formatting, payables/working-capital formulas, peer benchmark values.
Cross-output reconciliation	Verify that the same calculated value remains consistent across Size of Prize, Chat, generated narrative and final Output/deck.	Golden company opportunity values, bucket totals, working-capital values and override results should match everywhere they appear.
AI evaluation	Repeatable golden prompts with scoring, not exact-text assertions.	S&M explanation, G&A source lineage, bucket breakdown, generated peer rationale.
UI E2E	Only critical user journeys and behaviours that require the real UI.	Create target -> confirm peers -> Case for Change -> Size of Prize -> Output; refresh persistence; export smoke.
Browser / UI coverage	Run the critical partner journey and high-risk UI behaviours across the agreed supported browser matrix.	Target -> Peers -> Case for Change -> Size of Prize -> Output; real-time loading, Chat/source attachment, navigation and export behaviour.
Visual/export	Representative deck render checks and targeted manual visual review.	Data Coverage overflow, chart labels, footer, page numbering, exported PPT vs browser preview.

11. Environment and test-data strategy

Environment/data	Current understanding	QA approach
Current GLS Stage	Already showcased for GLS and expected to be abandoned after the current period.	Do not invest further regression effort unless specifically requested. Preserve existing evidence.
Future stable Stage / 50-partner release	Post-GLS planning indicates a stable Stage-type environment will support the first broader partner release.	Define smoke, golden regression, service/contract health checks and release evidence once the environment is confirmed.
GLS seeded golden data	Current eight-company demo/test dataset loaded separately (currently via Redis in the workshop explanation).	Keep as regression/evaluation baseline; do not treat it as proof of production data freshness/connectivity.
Production/live source data	Longer-term source data will move through the real data pipeline/data products and planned stable stores.	Add reconciliation/freshness checks when authoritative source connections are available.

Golden data handling

- Use the golden dataset as the stable baseline for deterministic regression and AI evaluation, with the test/evaluation logic referencing a versioned dataset identifier.
- Do not store protected/raw golden data directly in Git. Keep it in an approved controlled test-data location; the exact storage mechanism should be agreed with engineering/security.
- Keep only schemas, test logic, evaluation definitions and non-sensitive version/checksum references in the repository.
- Any change to the golden baseline should be reviewed and versioned so test results remain traceable.

Performance and accessibility baseline

- Measure the current full-run time for the golden companies.
- Measure time to first visible progress/result to create a baseline before formal performance targets are defined.
- Keep these results as the starting point for future performance comparisons.

12. How QA will execute a new OC ticket

#	Action	OC meaning
1	Read the story in product terms	Identify which journey step changes: Target, Peers, Case for Change, Size of Prize, Output or Chat.
2	Map the affected technical path	Identify BFF/service/data/event/agent layers touched by that story. Do not test only the visible screen.
3	Select relevant regression risks	Reuse the permanent risk scenarios above where they overlap with the change.
4	Choose the right data	Use the most relevant golden company/currency/sector/boundary case.
5	Design exact + AI checks separately	Assert deterministic numbers/state exactly; score generated narrative/rationale separately.
6	Run targeted exploratory checks	Refresh, repeated click, network loss, two tabs, pending analysis, stale responses and retry as relevant.
7	Reconcile downstream impact	If an earlier step changes, verify later Size of Prize/Output/Chat data where the dependency exists.
8	Attach concise evidence	Jira/report should show environment, data used, what passed, defects, observations, screenshots/video/API evidence and remaining risk.

13. QA readiness and sign-off

Ready for QA when

- The story states which Opportunity Catalyst screen/service behaviour is changing and has testable acceptance criteria.
- Required contract/schema changes are available if the story changes BFF/service/event payloads.
- The agreed post-GLS QA environment is available with the required feature flag and service versions.
- Required test data exists: seeded/golden company, business-unit mapping, source document or live-data reference as applicable.
- Developer smoke testing is complete and known limitations are stated.

QA complete when

- Story acceptance criteria pass in the agreed environment.
- Relevant deterministic calculations/state checks pass.
- Relevant AI/golden evaluations meet the agreed quality threshold.
- Relevant contract/integration and downstream regression checks pass.
- No unresolved blocker/high-risk issue remains unless explicitly accepted by the owner.
- Jira/report contains the tested data, evidence, defects/observations and any deferred risk.

Proposed QA release cycle

- PR / change validation: run the fastest relevant contract, service/API, calculation and targeted agent checks for the changed area.
- Merge / shared-environment validation: run integration checks and critical UI smoke for the affected journey.
- Pre-release validation: run the agreed golden regression, AI evaluation, cross-output number checks, critical E2E/browser coverage, multi-user/multi-tab scenarios, and applicable safety/performance/accessibility checks.
- Release decision: publish a concise QA summary showing what passed, known defects/risks, deferred items and whether the agreed release gate is met.

QA reporting and cadence

- PR/CI result: generated for each relevant change or pull request.
- Golden regression report: generated for release candidates and after changes that can affect calculations, prompts, models, tools or data mappings.
- AI evaluation report: generated when prompts/models/tools change and before a release.
- Daily QA update during active testing: short status covering passes, failures, blockers and major risks.
- Pre-release QA summary: one consolidated report for each release candidate, including release recommendation and remaining risk.

14. What QA should do during Sprint 5

Sprint 5 QA activity	Concrete output
Attend the workshops	Capture architecture/calculation/service ownership details that change how we test the product.
Refine this strategy after each relevant workshop	Especially calculations, UI/Chat, peer/narrative agents, BFF/case services, CI/CD, database/migrations and stage-to-main.
Prepare the golden regression pack	Document the eight seeded companies, expected deterministic values, known visual cases and standard Chat/AI questions.
Prepare contract/API regression	Turn previous malformed-response, peer-service and case/peer-state findings into repeatable automated checks.
Prepare AI evaluation scoring	Agree accuracy, grounding, completeness, relevance, consistency and hallucination criteria for Chat/rationales/narrative.
Map post-GLS backlog to QA risk	As stories are prioritised, assign data needs, affected services and required regression before execution starts.

15. Decisions still needed from the team

QA should not invent these answers. They need to be confirmed through the remaining workshops, story refinement or product/engineering decisions.

- Which environment becomes the official post-GLS QA / 50-partner validation environment, and what is its deployment cadence?
- Which JSON contracts are mandatory PR/build gates and which are only runtime checks?
- Who owns Business Unit mapping/rules and how should BU selection affect peer eligibility and Size of Prize?
- What are the final max-peer/min-peer and pagination/search rules after the Peer changes?
- What are the accepted AI-evaluation thresholds for Chat, peer rationale, Case for Change narrative and Recommended Actions?
- Which data source is authoritative for each major metric once production connections are enabled, and what freshness is acceptable?
- Confirm the remaining Audit/Evidence requirements not already covered by ADR-005, ADR-0040 and ADR-0045.
- What trace/correlation information will QA have access to for following UI -> BFF -> service -> event/agent failures?
- Confirm the final QA/release gate for the 50-partner rollout with Sandeep and Kasia.
- Confirm whether Nike and the other GLS target companies should also be included in the golden regression set.
- Confirm the supported browser matrix for the 50+ partner beta release.
- Confirm the approved golden-data storage/location and access model so protected raw data is not committed to Git.
- Confirm the final QA reporting cadence and owners for PR/CI, regression, AI evaluation and pre-release reporting.

This strategy should be updated when these decisions are made. Until then, the document deliberately avoids inventing detailed requirements that are not yet agreed.

Appendix A. Architecture reference from the workshop

Use this section as reference only. The main strategy above remains feature- and risk-driven.

Architecture topic	QA reference
Authentication / entry	Microsoft Entra handles login. Azure Front Door is the public entry point; traffic continues through Private Link into the internal Azure environment.
Hosting	The frontend is an SPA; services run in Azure Container Apps in a private subnet.
Experience BFF	Single frontend-facing backend entry point that orchestrates specialist services.
Data stores	PostgreSQL for changing application data; Blob for documents/search artefacts/Parquet; Redis for sessions/current GLS data; Snowflake planned for stable source data; Service Bus for async messages.
Agents	Agents run through Microsoft Foundry; use-case-specific agents are called by services or call services as tools. Model access is expected through Portkey.
Contracts	Central JSON schemas define service/event payloads and are expected to be checked in CI/build/runtime paths.
Repository areas	agents, contracts, services, tools/scaffolder, packages/entity-normalizer, utilities/search index builder, data product, database migrations and docs/decisions are separate concerns in the repo.

Appendix B. Inputs used

- Sprint 5 workshop plan and current housekeeping / knowledge-transfer direction.
- Post-GLS backlog: Target, Peers, Analysis/Size of Prize and Chat changes.
- Architecture workshop: BFF, service boundaries, data stores, contracts, real-time events, agents and repo structure.
- Existing OC QA evidence for Peer Selection, Peer Service, Peer Changes, Size of Prize, Output, Chat and Bain template conformance.
- Current eight-company seeded golden dataset and known expected results/visual behaviours.